

⚡ End-to-End Datenpipeline & KI-Caching-Architektur

1 News Ingestion

RSS / Atom Feeds oder Web-APIs werden per NewsSource-Adapter abgerufen und thematisch vorgefiltert.

DuckDB: news:rss [TTL]

2 KI-Graph-Extraktion

Pydantic-AI Agent zerlegt Text in Argumente (Claims) & Angriffsrelationen (Attacks). Kein Halluzinieren von URLs.

DuckDB: framework:key

3 Formale Inferenz

Mathematische Berechnung der Preferred Extensions nach Dung (1995) via Backtracking oder SAT-Solver.

DuckDB: extensions:naive

4 Topologie & Metriken

Graphentheoretische Scores (Akzeptanzrate), Streitachsen ($A \leftrightarrow B$) und Knotengrade (In/Out-Degrees).

In-Memory (Pure Functions)

5 KI-Synthese & Export

Zweiter Pydantic-AI Agent formuliert Perspektiventhesen. Automatischer Export mit Datumsstempel nach output/.

output/YYYYMMDD_*.md

📰 Konkretes Pipeline-Szenario: „Olympia-Bewerbung München“

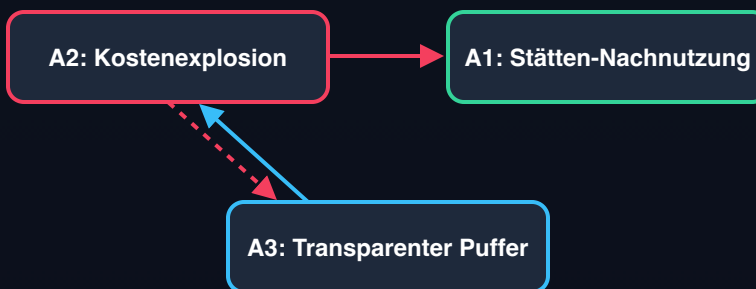
1. EXTRAHIERTE ARGUMENTE (A)

A1 (Wirtschaft): Nachnutzung bestehender Sportstätten stärkt Wirtschaft & Infrastruktur.

A2 (Finanzkritik): Kostenexplosionen belasten Steuerzahler unkalkulierbar.

A3 (Absicherung): Hoher Puffer & Transparenzregeln sichern vor finanziellen Risiken ab.

2. GRAPH-RELATIONEN (R)



A2 greift A1 an | A3 verteidigt A1 gegen A2 | A2 ↔ A3 bilden Streitachse

3. SYNTHETISIERTE PERSPEKTIVEN

Perspektive 1 (Pro): {A1, A3}
KI-These: Nachhaltige Chance für Sport und Stadt bei kontrollierbarem Haushaltsrisiko.

Perspektive 2 (Contra): {A2}
KI-These: Unkalkulierbare Kostenfallen übersteigen jeden potenziellen Nutzen.

🤖 Zweistufige KI-Verdrahtung

Pydantic-AI Nativ: Stufe 1 wandelt Freitext in streng typisierte ArgumentationFramework-Objekte. Stufe 2 verdichtet berechnete mathematische Extensions in verständliche Meta-Thesen.

uv run clashpy --solver naive

📋 Strukturelle Rollen

Mathematische Klassifikation nach Akzeptanz-Score über alle Extensions:

Basis-Konsens (Score = 1.0)

Umstritten ($0 < \text{Score} < 1$)

Isoliert / Verworfen (Score = 0)

🔒 Zero-Leakage Secrets

Vollautomatische Auflösung von API-Keys ohne Hardcoding:

- os.environ
- Lokale .env Datei
- Verschlüsselter OS Keyring (db.system.datahub)